

Coding Interviews for Cracked Rustaceans

A Zero-Cost Abstraction Obsession Anatomy

Arpan Pathak

Contents

<i>Foreword</i>	3
<i>Preface</i>	4
<i>The code</i>	5

Part I — The Cost Model

Part page	9
<i>1. What an Abstraction Costs</i>	10

Foreword

What you don't use, you don't pay for. What you do use, you couldn't hand code any better.

— Bjarne Stroustrup, *Foundations of C++*, ECOOP 2012

This book walks through one repository, `rust-interview-lab`, file by file. Each chapter takes a single file, states the problem it solves, prints the file, shows the state of the program after each step on a concrete input, and gives the cost of the operations it defines.

The code is printed as the repository holds it, documentation comments and tests included. Where a name in a listing would identify an employer or one of its products, a neutral name is printed in its place; the preface lists every substitution that was made.

The chapters can be read in order or one at a time. Each one stands alone, and each one ends with a program you can run.

Arpan Pathak

Preface

About this book

This book is a reading of one Rust codebase. Each chapter takes a single source file from the repository `rust-interview-lab`, states the problem the file solves, prints the file in full, follows its execution on a concrete input, and gives the cost of the operations it defines.

The subject is the reasoning that surrounds a program rather than the program alone: how a representation is chosen, what it costs to use, and which inputs it does not survive. Those are the questions an interviewer asks after the code is written, and the questions a reviewer asks before code is merged.

Who should read it

The chapters assume that you can read Rust. You should know what `Vec`, `HashMap`, `Option`, and `Result` are, and you should have written a few thousand lines of the language. Nothing else is assumed: every data structure the book uses is built in front of you, and every cost claim is either derived in the text or measured by a program printed in the text.

Two chapters carry the most weight. Chapter 12, on dynamic programming, is where the choice between a sentinel and an `Option` is worked out in full. Chapter 22, on lock poisoning, is where a failure that a program cannot prevent is handled deliberately. Both repay a second reading.

How to read it

Each chapter is a complete unit. It follows the same six sections, and closes with a section on the questions the chapter prepares you to answer and a list of the sources it rests on.

The problem. The input, the output, and the boundary cases that decide whether an answer is complete.

The algorithm. The mechanism the solution rests on, and why it is correct.

The implementation. The file as the repository holds it, including its documentation comments and its tests, followed by a reading of its decisions.

A worked example. The state of the program after each step on a concrete input, as a table or a diagram. Every trace was produced by running the code.

Cost. Time and space, with the condition stated in the same row when a bound holds only under an assumption.

Failure modes. The cases the code does not handle, the assumptions it makes about its input, and the resources it does not bound.

The chapters are grouped by subject. Chapters 1 to 13 cover the problems of the standard interview repertoire: array and string work, trees, graphs, recursion, and search. Chapters 14 to 18 cover linked structures and caches, in which the question is who owns what. Chapters 19 to 25 cover concurrency and the operating system, including one program that measures the cost of a system call. Chapter 26 covers the integration tests, which are the only place where the library's exported names are exercised the way a caller exercises them.

The code

The code in this book is the repository `rust-interview-lab`. To run it:

```
cd rust-interview-lab
cargo test                                # library tests, program tests, integration tests
cargo run --bin count_islands
cargo run --release --bin syscall_overhead
cargo run --bin mutex_poisoning
cargo run --bin shadowing
cargo run --bin singly_linked_list
cargo run --bin ll
cargo run --bin bst_clean
```

The listings reproduce the files as the repository holds them, with two exceptions, both mechanical.

Names that identify an employer or one of its products have been replaced with neutral ones. In the listings you will find `parse_device_count` where the repository names a product, `node-alpha`-style keys in the cache tests, `repl` and `replace` in the trie tests, and `rust_interview_lab` as the imported crate name. The comments speak of cloud SDKs and accelerator workloads. Nothing else about the code is altered, so a reader comparing a listing against the repository will find the same functions under different nouns.

One listing is completed rather than reproduced. The file printed in Chapter 18 ends with a test whose body stores a value and asserts nothing; it appears here with an assertion added, so that the test exercises the behaviour its name describes. Chapter 18 marks the place.

Conventions

A trace is a table with one row per step. Where a structure needs a picture rather than a table, boxes drawn inside a frame are values on the stack, and boxes to the right of the frame joined by an arrow are values on the heap. A solid arrow is a pointer that owns what it points at; a dashed arrow is a pointer that does not.

```
stack                                heap
+-----+                            +-----+
| variable: value | ----->         | owned value  |
+-----+                            +-----+
```

Cost tables use the standard notation. A bound that holds only under an assumption states the assumption in the same row:

Resource	Cost	Condition
Time	$O(n)$ expected	the hash function distributes keys evenly

Resource	Cost	Condition
Space	$O(n)$	up to n entries

A measured figure states the machine that produced it, because a measurement without its environment is not reproducible. The two programs that print measurements are given in full in Chapters 24 and 26 of the repository's own documentation, so a reader can obtain the same figures locally.

Acknowledgments

The claims about the Rust language and its standard library rest on the documentation the Rust Project Developers maintain: the *Rust Book*, the *Rust Reference*, the *Rustonomicon*, the standard library reference, and the API Guidelines. The algorithms are attributed in each chapter and listed with their sources in the References. The quotation in the foreword is Bjarne Stroustrup's statement of the zero-overhead principle, and the four that follow it in the chapters are attributed where they appear.

The Code

Layout

The repository holds one Cargo package containing a library, seven programs, and an integration test file.

```
rust-interview-lab/
├── Cargo.toml
├── README.md
├── src
│   ├── lib.rs                                the crate root and its re-exports
│   └── problems                             the module list
│       ├── mod.rs
│       ├── backtracking.rs                 Chapter 13
│       ├── binary_search.rs               Chapter 9
│       ├── binary_tree.rs                 Chapter 7
│       ├── dp.rs                           Chapter 12
│       ├── graph_topology.rs               Chapter 11
│       ├── linked_list.rs                 Chapter 14
│       ├── lru_cache.rs                   Chapter 17
│       ├── lru_cache_easy.rs               Chapter 18
│       ├── merge_intervals.rs              Chapter 5
│       ├── min_stack.rs                   Chapter 3
│       ├── rate_limiter.rs                 Chapter 20
│       ├── sliding_window.rs               Chapter 4
│       ├── state_machine.rs                Chapter 19
│       ├── top_k_frequent.rs               Chapter 6
│       ├── trie.rs                         Chapter 10
│       ├── two_sum.rs                      Chapter 1
│       ├── valid_parentheses.rs            Chapter 2
│       └── worker_pool.rs                  Chapter 21
├── bin
│   ├── bst_clean.rs                       Chapter 8
│   ├── count_islands.rs                   Chapter 23
│   ├── ll.rs                              Chapter 16
│   ├── mutex_poisoning.rs                 Chapter 22
│   ├── shadowing.rs                       Chapter 25
│   ├── singly_linked_list.rs               Chapter 15
│   └── syscall_overhead.rs                 Chapter 24
└── tests
    └── problems.rs                         Chapter 26
```

`src/problems/hash_map.rs` is an empty file. Nothing in the crate refers to it.

Modules and programs

Nineteen of the files are library modules. Each holds one problem, one type, or one pair of a type and its policy, and each ends with a `#[cfg(test)] mod tests`, so the list printed by `cargo test` reads as an index of this book.

Seven files are programs. `src/bin/*.rs` are separate crates that link the library, and each has a `main` function that runs something visible: a measurement, a traversal, a panicking thread, a list. Three of them are not library modules at all — `count_islands`, `syscall_overhead`, and `shadowing` stand alone — and the remaining four demonstrate a library type.

What the files have in common

Four habits run through the code, and they are stated here once so that the chapters need not repeat them.

Enumerations instead of flags. Where a value has a fixed set of states, the code uses an `enum` and a `match` with no fall-through arm, so that adding a state is a compile error at every place that must handle it. `WorkloadState` in Chapter 19 and `ApiError` beside it are examples.

Option and Result instead of sentinels. Absence is `Option` and failure is `Result`, so the type says which. Chapter 12 keeps a sentinel inside the dynamic program's table and translates it at the boundary, and a caller never sees it.

Ownership written out. The linked structures in Chapters 14 to 16 use three different answers to "who owns the next node" — `Box`, `Rc` with `Weak`, and a vector of indices — and each answer is visible in the type rather than in a comment.

Tests beside the code. A unit test that reaches a private item is a licence to change that item. The integration tests in Chapter 26 exercise the same behaviour through the public API, which is what checks the re-exports.

Part I — The Cost Model

1. What an Abstraction Costs

A field that costs nothing

The repository's singly linked list stores its successor as `Option<Box<ListNode>>`. On a machine with 64-bit addresses that field is eight bytes wide, which is the width of a bare `Box<ListNode>` in the same position. The `Option` wrapper is free: no tag byte, no padding, no second word. The empty case is encoded in the pointer itself, because a boxed value is never null, so the all-zero address is available to mean `None`.

Change one word of that type and the story changes. `Option<u32>` occupies eight bytes, against four for a `u32` on its own. Every `u32` bit pattern is a legal `u32`, so there is no spare encoding for the empty case and the compiler adds a four-byte tag, which alignment then rounds up to eight.

type	size	alignment	why
<code>Box<ListNode></code>	8	8	one pointer
<code>Option<Box<ListNode>></code>	8	8	the null pointer is the empty case
<code>u32</code>	4	4	one word
<code>Option<u32></code>	8	4	no spare encoding, so a tag is added
<code>&[i32]</code>	16	8	a pointer and a length
<code>Vec<u32></code>	24	8	a pointer, a length, and a capacity

These figures are for a 64-bit target and were produced by a program run on the machine that built this book. They are not properties of the language. On a 32-bit target each pointer is four bytes and every one of the numbers above changes. What survives the change of target is the reason in the last column: the layout follows from whether the type has an unused bit pattern, and that is a property of the type.

So a design question as ordinary as "should this field be `Option<u32>` or `Option<NonZeroU32>`" has a measurable answer, and the answer is four bytes per value. The linked list gets the free version without being asked to, because it stores a pointer. The `MinStack` pays the tagged version twice over, in two `Vec<i32>` fields, and pays nothing at all for the option because `Vec` handles emptiness through its length.

Four budgets

An abstraction spends from one of four budgets, and a claim about overhead is meaningless until it names which one.

Budget	Measured in	Typical mechanism that spends it
Instructions	cycles per operation	one indirect call per element through a trait object
Memory	bytes per value, bytes per live object	a tag byte added to an enumeration, an entry per access in a log
Stack	bytes per frame, frames per call	recursion whose depth follows the input
Compilation	seconds per build, bytes per artefact	one copy of a generic function per set of type arguments

The first two are the ones a candidate is usually asked about. The third appears when a structure is deep, and the fourth when a program is large. They are not interchangeable: an abstraction can be free in instructions and expensive in memory, which is exactly what the linked list is, and it can be free in memory and expensive in instructions, which is what a boxed iterator is.

What the compiler removes

The mechanism behind most zero-cost claims is monomorphisation. The *Rust Book* defines it as "the process of turning generic code into specific code by filling in the concrete types that are used when compiled", and the consequence is mechanical: a generic function with one call site compiles to one function, which the optimiser can then inline into its caller.

The repository relies on this everywhere. `map_order<T, R, F>` in Chapter 21 takes a closure as a type parameter, so the pool's worker loop contains a direct call to the caller's closure rather than an indirect call through a function pointer, and the closure's body can be optimised in place. The iterator chains in the search chapter compile to loops because `Map`, `Copied`, and `Filter` are types and each one is named at the call site.

The same mechanism is why the fourth budget exists. Monomorphisation is paid for with code: a generic function with forty call sites containing forty different types produces forty copies of the compiled function. On a large program that shows up as build time and binary size, and the trade is real enough that the standard library itself avoids generic instantiations in a few places for this reason.

What it cannot remove

Four things survive every optimisation level, and a candidate who can name them before being asked is answering the question the interviewer is building towards.

A bounds check on an index that the optimiser cannot prove is in range stays in the generated code. So does a `match` on a discriminant, a pointer chase through a box, and the unwinding tables that a reachable `panic!` requires.

Type erasure removes the ability to inline. Replacing a concrete type with `dyn` turns each call into a load of an address from a vtable followed by an indirect call, and the optimiser loses the body it would have inlined. The price is per call, not per program, so it depends entirely on how often the call happens.

Work that has to leave the process cannot be optimised at all. The lab's system call measurement is the clearest example in the book: ten million calls to `getpid` take 2.076 seconds on the machine that ran it, or 207.6 nanoseconds per call, and no amount of inlining touches that number because the time is spent in the kernel. That measurement is discussed in Chapter 19.

How to read the numbers in this book

Two kinds of number appear in the chapters. Facts about the language or the standard library are stated as facts and cited, with the version attached where a function was stabilised in a particular release. Measurements are given with the machine that pro-

duced them and the number of repetitions, because a measurement without its environment cannot be checked, and the two are not interchangeable.

Where a bound is conditional, the condition appears beside it. The expected constant-time lookup of a hash map is the clearest case: it holds while the hash function distributes keys evenly, and the standard library's default hasher is randomly seeded precisely so that an attacker cannot choose the keys that break the assumption. A program that installs its own hasher gives that up, and the bound becomes a hope.

The repository

Every chapter that follows reads one file from `rust-interview-lab`, prints it, and works out what it costs. The book is organised by subject rather than by file, because the interesting questions — where a value lives, what a move does, how deep a recursion goes, what a lock costs — are answered by comparing several files. Chapters name the files they draw on, and the code in the listings is the repository's code, with the substitutions described in the preface.

References

- The Rust Book, [Performance of code using generics](#), on monomorphisation.
- The Rust Book, [Using trait objects that allow for values of different types](#), on dynamic dispatch.
- The Rust Reference, [Type layout](#), on size, alignment, and the niche optimisation.
- Standard library, `std::mem::size_of` and `std::mem::align_of`.
- Standard library, `HashMap`, on the randomly seeded default hasher.